

ProGen: Projection-Based Adversarial Attack Generation Against Network Intrusion Detection

Minxiao Wang, Ning Yang^{ID}, *Member, IEEE*, Nicolas J. Forcade-Perkins,
and Ning Weng^{ID}, *Senior Member, IEEE*

Abstract—Adversarial attacks, widely recognized as significant threats to machine learning (ML) models in computer vision and natural language processing, can have more severe consequences when targeting ML-based Network Intrusion Detection Systems (NIDS). These attacks, characterized by data manipulation, necessitate a focused investigation grounded in the unique attributes of the data and practical constraints inherent to the target scenario, as opposed to indiscriminately applying methodologies borrowed from other domains. Since network traffic is complex unstructured data, ML models are commonly used in existing studies to explore how perturbations can defeat ML-based IDS. However, two challenges persist in the realm of traffic-space adversarial attack generation. First, raw traffic data cannot be directly input into ML models. Second, determining the appropriate perturbation scale and direction is challenging, particularly in the case of multi-class NIDS. In this work, we propose a projection-based adversarial attack generation framework, ProGen, to address these two challenges. ProGen is inspired by two observed characteristics of the NIDS scenario: flexible representation and clear objective. ProGen uses a basic feature sequence (BFS) space to represent network traffic in a way that aligns with realistic requirements. To achieve a clear objective, ProGen utilizes a traffic space generative adversarial network (GAN) to approximate distribution mapping between malicious traffic and benign traffic. To better apply the generative model for adversarial attacks, we further design constraints to preserve the functions of the adversarial traffic. We've successfully demonstrated the effectiveness of ProGen on six common ML models using the CSE-CIC-IDS2018, CIC-IDS-2017, and UNSW-NB15 datasets; however, we're yet to validate these findings in real network environments. We visualize the generated distributions of the BFS elements to illustrate the projecting effect under the designed realistic constraints. The results of attack effectiveness tests show that attacks generated from ProGen can significantly reduce the detection performance across different ML models.

Index Terms—Intrusion detection, security system evaluation, machine learning security.

I. INTRODUCTION

THE study of the reliability of machine learning methods is crucial, particularly when they are employed as part of security mechanisms such as NIDSs. Given that many ML-based NIDS approaches have been presented to improve NIDS effectiveness [2], the vulnerability of ML methods to adversarial attacks is receiving increasing attention [14]. Adversarial attacks, which exploit the inherent vulnerabilities of ML and deep learning (DL) models, pose challenges to various DL applications, such as Computer Vision (CV) [11] and Natural Language Processing (NLP) [20]. An even graver concern arises when considering adversarial attacks in security-related domains. In such scenarios, the ML models may fail to reinforce security and adequately mitigate risks, leading to potentially dire consequences. For example, adversarial evasion attacks against ML-based Network Intrusion Detection Systems (NIDSs) can significantly undermine the fundamental objectives, which are identifying and thwarting malicious network attacks, thereby allowing malicious network traffic to go undetected and inflict damage.

In addition to the potential consequences, adversarial evasion attacks against NIDSs also exhibit notable distinctions from those in CV and NLP in two other significant aspects, referred to *flexible representation* and *clear objective*. First, the distinctive data structure of network traffic introduces fundamental disparities in the methods for generating these attacks when compared to the image data in CV or textual data in NLP. For instance, varying network traffic analysis methods can *flexibly* use different traffic representing formats, such as different feature sets, according to their specific needs. Second, unlike randomly inducing misclassification in other domains, adversarial attacks against NIDSs possess a *clear objective*: to mimic benign network traffic and evade detection [12], [28].

Over the last few years, growing interest has been in investigating adversarial attacks against ML-based NIDSs. These proposed attacks typically fall into two primary categories: feature-space attacks and traffic-space attacks. However, a notable limitation of feature-space attacks has emerged, primarily stemming from their impracticality [1], [12]. This impracticality arises from the necessity of possessing

Manuscript received 31 October 2023; revised 1 March 2024 and 4 May 2024; accepted 4 May 2024. Date of publication 16 May 2024; date of current version 23 May 2024. This work was supported in part by the Dr. Yang's Startup Fund and in part by NSF under Award 2018919. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Dusit Niyato. (*Corresponding author: Ning Yang.*)

Minxiao Wang and Ning Weng are with the Computer Engineering Program, School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University, Carbondale, IL 62901 USA (e-mail: minxiao.wang@siu.edu; nweng@siu.edu).

Ning Yang is with the Information Technology Program, School of Computing, Southern Illinois University, Carbondale, IL 62901 USA (e-mail: nyang@siu.edu).

Nicolas J. Forcade-Perkins was with the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University, Carbondale, IL 62901 USA. He is now with Texas Instruments, Dallas, TX 75243 USA (e-mail: N-Forcade-perkins@ti.com).

Digital Object Identifier 10.1109/TIFS.2024.3402155

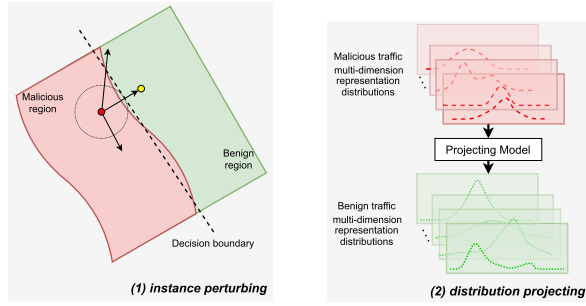


Fig. 1. Comparison between instance perturbing-based and distribution projecting-based adversarial traffic generation. The perturbing-based method adds perturbation noise to move each instance across the decision boundary of the target model. The projecting-based method aims to transport the data from the source distribution to the target distribution by solving the distribution mapping problem between groups of benign and malicious traffic.

knowledge about the specific features employed by the target NIDS, as perturbing these features may potentially lead to conflicts within the Transmission Control Protocol/Internet Protocol (TCP/IP) stack or even result in the loss of malicious functionality.

Furthermore, most existing approaches, both feature and traffic-based, followed the CV’s adversary philosophy, which is introducing noise into the original data sample with the intention of crossing the decision boundary of the target ML model. However, considering the unique *clear objective* property of evasion attacks against NIDSs, we believe the adding noise philosophy (we refer it to instance perturbing, as shown in the left part of Fig. 1) is not necessary to be followed in the field of NIDSs for the following two reasons. First, it is worth noting that multiple-class NIDSs are commonplace. In such scenarios, merely inducing random misclassification does not guarantee a successful evasion from NIDSs. Second, the adversarial malicious traffic is constrained by communication legality and functionality preservation instead of the noise scale.

In this paper, we present a projection-based traffic space adversarial evasion attack generation method. Our adversary philosophy is referred to as distribution projecting, which is shown in the right part of Fig. 1. We first study the uniqueness of the NIDS field and the relation between feature and traffic space to decide on using the **basic feature sequence space (BFS)** (in Section IV-A) as our projecting space. To fit the unique *flexible representation* and *clear objective* characteristics of NIDSs, we design an attack generation framework ProGen, which takes advantage of the advanced traffic space generative adversarial network (GAN) (DoppelGANger [19]) for autonomously projecting the original malicious traffic to the benign traffic space and preserving its communication legality and functionality. To demonstrate the effectiveness of ProGen, we conducted experiments using the 3 datasets. We evaluate the impact of ProGen on six commonly used ML models, showcasing its potential to enhance the robustness of NIDS systems.

The contributions of this work are as follows:

- We present projecting-based adversarial attack generation for network traffic. First, we present a basic feature

TABLE I
NOTATIONS IN PROBLEM FORMULATION

Notation	Description
f	symbols represent traffic feature
$\hat{\mathcal{A}}(\cdot)$	the adversarial operator
$\mathcal{D}(\cdot), \mathcal{D}'(\cdot)$	the target NIDS classifier, the surrogate discriminator
$\mathcal{F}, \mathcal{M}, \mathcal{S}$	basic feature sequence representation, meta-information, basic feature measurement series
$\mathcal{F}_a, \mathcal{F}_b, \mathcal{F}_{adv}$	symbols represent attack, benign, and adversarial traffic
$\mathcal{C}_s, \mathcal{C}_t$	symbols represent source and target classes
Ω_s, Ω_t	the source and target collected traffic sets
$\mathcal{P}_s, \mathcal{P}_t$	the source and target traffic distributions

sequence traffic representation format, which can overcome the limitations while incorporating the advantages of both the traditional feature and traffic representation space. Then, within this representation, we present a projecting-based adversarial operator by solving the distribution mapping problem between groups of benign and malicious traffic.

- We design ProGen, an adversarial evasion traffic generation framework. By utilizing a traffic space GAN, ProGen learns an approximated transport map between benign traffic distribution and malicious traffic distribution by training with collected benign traffic from the target network environment. To ensure the realism of the generated traffic space GAN, we introduced two constraints: (1) the timestamp shift pre-processing, which protects the GAN from misleading cross-flow temporal information; (2) the perturbation loss, which ensures that the perturbed traffic remains realistic and functional. Then we evaluate the realism by showing the distribution of the generated BFS with different constraints in Sec. V-B.
- We demonstrate the effectiveness of ProGen by using the adversarial samples generated under different constraints to evaluate six common ML models trained on the CSE-CIC-IDS2018, CIC-IDS-2017, and UNSW-NB15 datasets. (However, we’re yet to validate these generated adversarial samples in a real network environment.)

In the remainder of this paper, we provide the necessary background in Section II. We define the threat model of our realistic adversarial attack in Section III. We present the details of our ProGen framework in Section IV. The evaluation results and analysis are presented in Section V. The highly related works are introduced in Section VI. Finally, in Section VII, we offer our concluding remarks and discuss future work.

II. BACKGROUND

In this section, we will provide an overview of the background related to adversarial attacks against NIDSs and explore how they differ from adversarial attacks in other domains.

A. Adversarial Attacks Against NIDS

To illustrate the general definition of adversarial attacks against NIDS, we first use the feature space adversarial attacks against NIDS as an example. Considering an operator $\mathcal{A} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ aims to fool a NIDS classifier $\mathcal{D} : \mathbb{R}^d \rightarrow \mathbb{R}$ by

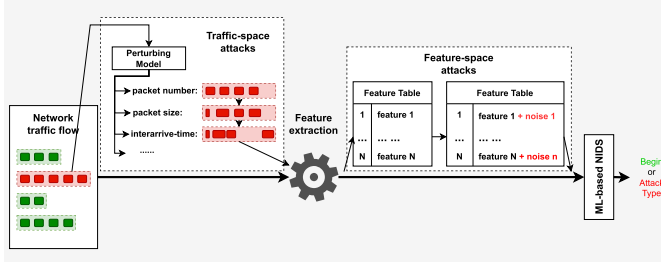


Fig. 2. Overview of traffic-space, and feature-space perturbing-based attacks in the workflow of ML-based NIDSs.

perturbing a given network traffic flow, whose feature space representation is $f_0 \in \mathbb{R}^d$, belongs to class C_s .

$$\begin{aligned} f_{adv} &= \mathcal{A}(f_0) = f_0 + n_0, \\ \mathcal{D}(f_{adv}) &= C_t \neq C_s = \mathcal{D}(f_0), \end{aligned} \quad (1)$$

where n_0 is the perturbation noise and the perturbed traffic feature vector f_{adv} will cause the NIDS classifier $\mathcal{D}$ to misclassify it as C_t . In an NIDS scenario, C_s is the original malicious class and the C_t is benign.

Based on the definition in Equation (1), we notice two degrees of freedom for various existing methods, which are the traffic representation space and the adversarial operator $\mathcal{A}(\cdot)$. With different traffic representations, the attack methods can be categorized into feature-space attacks and traffic-space attacks. With different conditions for finding a satisfied adversarial operator $\mathcal{A}(\cdot)$, the attack methods can be categorized into black box, gray box, and white box attacks. For example, the Fast Gradient Sign Method (FGSM), as a white box attack, takes the parameter ω of classifier $\mathcal{D}$ as one of the inputs for calculating $n_0 = \eta \cdot \text{sign}(\nabla_f J(f_0; \omega))$ in $\mathcal{A}(\cdot)$ [11].

Therefore, in this work, we will present our method by respectively introducing our design for the traffic representation space $\mathcal{F}$, see Section IV-A and the adversarial operator $\mathcal{A}(\cdot)$, see Section IV-B.

B. Feature Space and Traffic Space

Given that many ML-based NIDS approaches have been presented to improve NIDS effectiveness [2], the vulnerability of ML methods to adversarial attacks is receiving increasing attention [14]. Existing NIDS adversarial attack methods typically have two primary branches, which are feature-space attacks and traffic-space attacks. How they work in a general NIDS scenario is shown in the left part of Fig. 2.

A general ML-based NIDS working pipeline is represented in Fig. 1. First, the network traffic is captured into traffic trace data. Then, the traffic trace undergoes the feature extraction process to represent the traffic in a format, which is beneficial to distinguish the malicious traffic and its attack type. Finally, the extracted features are fed into an ML-based NIDS model to classify the traffic as benign traffic or what types of networking attacks.

Feature-space attacks occur after the feature extraction stage, as shown in Fig. 2. The feature-space attacks draw inspiration from the field of CV [14] to directly add noise to the extracted feature vectors. The *advantages* of feature-space attacks are listed as follows,

- (a) *The same space with NIDSs*: Given that ML-based NIDSs work in feature space, it is convenient to directly explore the weakness in feature space by adding noise.
- (b) *Easy to learn*: Given that more ML-based methods are adopted to generate adversarial attacks against NIDSs, the feature vector format makes it easy for ML models to learn the pattern.

However, many researchers have noted the difference in pre-processing between NIDSs and CVs, that the extracted features are not independent [22], [31]. Therefore, the feature-space attacks have some inherent *shortcomings* as follows,

- (a) *Risk of feature conflicts*: Perturbing traffic feature vectors has a high-degree risk of causing conflicts among multiple feature dimensions, which means that the perturbed feature vector does not represent any actual network traffic.
- (b) *Extra cost of verification*: To overcome that risk, extra verification has to be applied to make sure the perturbed feature vector is legal.

Traffic-space attacks occur before the feature extraction stage, as shown in Fig. 2. By directly perturbing raw traffic while adhering to the TCP/IP protocol stack rules. Traffic-space attacks naturally circumvent the risk of feature conflicts. Hence, the biggest *advantage* of traffic-space attacks is the realistic and practical perturbation, based on which adversarial attacks can be launched against real-world NIDS scenarios. However, perturbing the raw network traffic is a challenge for the following reasons:

- (a) *Unpredictable length*: The unpredictable length (including packet number) of network traffic flow makes it unsuitable for most existing ML models, which normally take fixed-shape inputs.
- (b) *Two levels information*: Raw network traffic consists of the flow level and the packet level information, both of which need to be perturbed.

Given those challenges, existing methods [12], [21], [29], [33] normally solve the traffic perturbation by using several pre-defined operations, such as (1) timestamp modification, (2) packet payload padding, (3) protocol layer modification, and (4) crafted packet injection, etc. However, in this case, the perturbing models become the “switch button” of the pre-defined operations instead of an optimizable noise generation model, like in the feature-space attacks.

C. Realistic Attacks

In the field of NIDSs, the realistic degree of adversarial attacks is crucial for the arms race between attackers and defenders. For attackers, the realistic degree refers to the feasibility of adversarial attacks. Only the adversarial attacks, which are feasible to launch in real-world scenarios, pose threats to the robustness of ML-based NIDSs. For defenders, the realistic degree indicates the possibility of threat. Given the limited resources, the security mechanisms should always focus on dealing with realistic threats instead of impossible or unlikely issues. Therefore, we believe that studying realistic adversarial attacks against NIDSs will bring more benefits to the robustness of ML-based NIDSs.

Instead of using the general black/gray/white box, we discuss the realistic adversarial attacks against NIDS based on the specific potential required assumptions and conditions for launching attacks. **Three realistic aspects** that influence generating adversarial traffic are identified as 1) Knowledge of the target system; 2) Access to the target system; and 3) Functions of generated adversarial traffic.

Knowledge of the target system is denoted as the attackers' knowledge about the target ML-based NIDS. Existing attack methods are designed under some assumptions, such as

- (a) *Knowledge of ML models*: Some adversarial attacks impractically assume knowing everything about the ML model of the target NIDS. For instance, the gradient-based attacks [6], [15] need to fully know the target model, even reproduce the model, to calculate the gradient for attacking.
- (b) *Knowledge of feature extraction*: Knowing the feature set employed by the targeted NIDS can aid attackers in determining the perturbations necessary for generating adversarial traffic.

Access to the target system refers to the capability essential to launch the designed attack method in NIDS scenarios, allowing interaction with the target system.

- (a) *Access to the target network*: Considering an NIDS scenario, access to the target network environment is the premise of accessing any parts of the target NIDS.
- (b) *Access to the feature extraction*: In feature-space attacks, having the capability to access the feature extraction component is essential for introducing perturbing noise to the extracted features.
- (c) *Access to the reaction of ML model*: For most reinforcement learning (RL)-based methods, having the capability to the reaction made by the target NIDS.

Functions of generated adversarial traffic are defined as the preservation of the functionality as malicious network traffic. No matter the feature or traffic-space methods, the perturbed traffic representation should preserve the following functions:

- (a) *Communication function*: The perturbed feature vectors should adhere to the TCP/IP protocol stack rules. The perturbed traffic should be able to transport in the network.
- (b) *Malicious function*: The perturbed traffic or the traffic represented by the perturbed feature vectors should maintain their malicious functions.

In summary, when it comes to the first two aspects—1) Knowledge of the target system and 2) Access to the target system—the higher the level of these requirements for an attack, the less realistic the attacks become. Conversely, in the case of 3) Functions of generated adversarial traffic, the higher the level achieved, the more realistic the attacks become.

III. THREAT MODEL

In this study, we consider a scenario in which an attacker plans to send malicious traffic to attack a target network environment protected by an ML-based NIDS. At the same time, the attacker uses adversarial examples to evade the detection

of ML-based NIDS. We present a realistic adversarial attack threat model against NIDSs by following the pre-defined three aspects in Section II-C.

First, we assume the attacker lacks detailed knowledge about the target NIDS ML model and extracted feature set.¹ However, the attacker, who is also a network attacker, has knowledge about what features are usually used for intrusion detection. Unlike the CV domain, in which many published standard models and trained checkpoints are open-access online, the ML-based NIDS study and applications are too customized to have a universal baseline model. Hence, we believe it is reasonable to assume limited knowledge.

Second, we assume the attacker is only able to access the target network and observe some reaction of the target NIDS, but not the feature extraction component because it is an inner part of the target NIDS. Because the attacker is also a network attacker, it is fair to assume the attacker has already established a foothold in the target network [1] to observe the benign traffic [28].

In addition, the manipulation of network attack traffic or the perturbation of features should also maintain the adversarial malicious traffic's communication and malicious functionality [1], [5], [29].

IV. ProGen FRAMEWORK

In this section, we present our ProGen framework for generating projecting-based adversarial traffic. In ProGen, we first represent network traffic in BFS format in Section IV-A. Then, we introduce the design of our adversarial traffic GAN, which is based on DoppelGANger, in Section IV-B. Finally, we further add extra constraints for training the traffic GAN to generate adversarial malicious traffic with preserved functionality in Section IV-D.

A. Traffic Representation

As previously discussed, in many ML-based network traffic analysis studies, including NIDSs, the options for traffic representation formats are highly flexible. This *flexible representation* affords us the opportunity to leverage the advantages of both feature and traffic-space methods to the fullest extent possible.

1) *Basic Feature Sequence (BFS) Space*: Based on our investigation in Section II-B and II-C, we can condense the two most significant advantages that traffic representations should possess for realistic adversarial traffic generation. First, the representation should be able to be directly fed into ML models to achieve good learning performance. Second, the representation has intrinsic immunity from the risks of perturbations, specifically easily preserving the perturbed traffic's communication and malicious functions.

To achieve those advantages, we present the traffic representation used in this study, named BFS.

$$\mathcal{F} : \begin{cases} \mathcal{M} &= \{\mathbf{ip}_{src}, \mathbf{ip}_{dst}, \mathbf{port}_{src}, \mathbf{port}_{dst}, \mathbf{proto}, \mathbf{id}\} \\ \mathcal{S} &= \{\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_N, \mathbf{Pad}_{N+1}, \dots, \mathbf{Pad}_{max}\}, \end{cases} \quad (2)$$

¹Denotes a set of handcraft features that is formatted as feature vectors for ML-based NIDS model classification.

where $\mathcal{F}$ represents a bidirectional network traffic flow, which can be identified by the meta-information $\mathcal{M}$ referred to the five tuples set of “source IP”, “destination IP”, “source port”, “destination port”, and “protocol”. In addition, a traffic ID, id , is used to distinguish the sub-flow with the same five tuples over time. The traffic $\mathcal{F}$'s behaviors are contained in the series of N transmitted packets $\mathcal{S}$, which is sorted by the timestamp. The last packet $\mathbf{P}_N$ is followed by zero padding to make sure $\mathcal{S}$ has fixed length max . Where $\mathbf{P}_i$ is denoted a basic feature vector for the i th packet. In this study, $\mathbf{P}_i$ is defined as

$$\mathbf{P}_i = \{\mathbf{t}_i, \mathbf{d}_i, \mathbf{h}_i, \mathbf{p}_i, \mathbf{f}_i, \mathbf{w}_i\}. \quad (3)$$

The six basic features; $\mathbf{t}_i, \mathbf{d}_i, \mathbf{h}_i, \mathbf{p}_i, \mathbf{f}_i, \mathbf{w}_i$, respectively represent the information “timestamp”, “packet direction”, “header length”, “payload length”, “flag”, and “TCP windows” carried on the i th packet.

The BFS format overcomes specific limitations while incorporating the advantages of both feature-space and traffic-space methods discussed in Section II-B. This format encapsulates traffic information on two levels: $\mathcal{M}$ represents the flow level, and $\mathcal{S}$ represents the traffic level. By converting network traffic into this BFS format as described in Equation (2), we can directly feed it into ML models for numerical computations. Additionally, as the basic features are independently recorded in each packet, any perturbations to these features do not result in conflicts among them.

2) *Case Study on BFS*: To figure out the specific relationship among BFS, feature, and traffic space representations, we present a case study of feature extraction on the CSE-CIC-IDS2018 dataset [27]. In the CSE-CIC-IDS2018 dataset, CICFlowMeter is used for traffic analysis and feature extraction. The CICFlowMeter tool calculates 75 features to detect if each traffic flow is benign traffic or malicious.

As indicated in Table XV, all the extracted features in CSE-CIC-IDS2018 can be computed using the BFSs described in Equation (3). To facilitate feature analysis, we initially classify these features into three distinct groups: bi-directional, forward direction, and backward direction, depending on the directional context they address. If a feature lacks corresponding counterparts in other directions, an empty space is retained. Moreover, we categorize these features into seven different feature contexts, as illustrated in the first column of Table XV, based on the underlying basic features they utilize. For example, features related to packet size are computed using the payload length BFS denoted as $\mathbf{p}_i$, with $\mathbf{p}_i$ being filtered differently based on the packet direction BFS $\mathbf{d}_i$ for various directional groups. Obviously, the features in the same row or in the same feature context group are highly correlated. Perturbing the features without taking into account these correlations can lead to conflicts among multiple feature dimensions, resulting in the perturbed feature vector no longer accurately representing actual network traffic.

B. Distribution Mapping

Adversarial evasion attacks against NIDSs have a *clear objective*, which is to mimic benign network traffic and evade detection. As targeted adversarial attacks, the success perturbation from the source class $\mathcal{C}_s$ to the target class $\mathcal{C}_t$ means that

the attacker needs to figure out both the perturbing distance scale and direction. However, within the realistic threat model defined for NIDS scenarios (as detailed in Section III), solving the appropriate perturbation noise n_0 for a given traffic $\mathcal{F}_0$ demands a considerable number of probing attempts to identify the correct direction and scale. Excessive probing, however, triggers alerts in the targeted NIDS, leading to the failure of the attack. To address this challenge, we introduce a projection-based method for generating adversarial network traffic.

1) *Projecting-Based Attack*: Consider an attacker plans to launch a type of network attack against a network environment, that is protected by an ML-based NIDS denoted as $\mathcal{D}(\cdot)$. The attacker prepared a set of malicious traffic denoted as Ω_s , for the attack traffic flow $\mathcal{F}_j^a \in \Omega_s$, the NIDS will classify them as $\mathcal{D}(\mathcal{F}_j^a) = \mathcal{C}_s$, where $\mathcal{C}_s$ is class name for the network attack. To bypass the NIDS, an adversarial operator $\mathcal{A}(\cdot)$ can manipulate the original traffic to cause the NIDS misclassification as $\mathcal{D}(\mathcal{A}(\mathcal{F}_j^a)) = \mathcal{C}_t$, where class $\mathcal{C}_t$ is the class name for benign traffic.

In this work, we solve the problem of the adversarial operator $\mathcal{A}(\cdot)$ in a distribution mapping perspective. From a distributional standpoint, Kulinski and Inouye [17] view distribution shift as transportation that the data samples in a source distribution $\mathcal{P}_s$ move to a target distribution $\mathcal{P}_t$. A distribution transport map $\mathcal{T}$ can illustrate the relationship between two distributions is defined as,

$$\mathcal{T} : \mathcal{P}_s \rightarrow \mathcal{P}_t. \quad (4)$$

Our projecting-based method aims to find an adversarial operator $\mathcal{A}$, which approximates to $\mathcal{T}$ in Equation (4). To achieve that, we need more information about the distributions $\mathcal{P}_s$ and $\mathcal{P}_t$. For the source distribution $\mathcal{P}_s$, the attacker has already prepared the attack traffic flow $\mathcal{F}_j^a \in \Omega_s$, where $\Omega_s \subset \mathcal{P}_s$. For the target distribution $\mathcal{P}_t$, we assume within the threat model that the attacker, also a network attacker, can collect a set of benign traffic flows referred to as $\mathcal{F}_j^b \in \Omega_t$ from the targeted network environment. The collected benign traffic set $\Omega_t \subset \mathcal{P}_t$ is a subset of benign traffic distribution $\mathcal{P}_t$. Therefore, the adversarial generation problem can be approximately formulated as solving

$$\begin{aligned} \arg \min_{\mathcal{A} : \Omega_s \rightarrow \Omega_t} & \mathbb{E}(\mathcal{C}(\mathcal{A}(\mathcal{F}^a), \mathcal{F}^b)) + \mathcal{L}(\mathcal{A}(\mathcal{F}^a), \mathcal{F}^a) \\ \text{s.t. } & \mathcal{F}_j^a \in \Omega_s, \mathcal{F}_j^b \in \Omega_t, \end{aligned} \quad (5)$$

where $\mathcal{C}(\cdot, \cdot)$ is the cost function measuring the difference between attack and benign traffic and $\mathcal{L}(\cdot, \cdot)$ is the constraint loss for preserving attacks' malicious function.

However, solving the optimization problem in Equation (5) is challenging for the following reasons. First, it is hard to define a cost function $\mathcal{C}(\cdot, \cdot)$ to measure the traffic similarity in the BFS representation space defined in Equation (2). Second, searching all possible mapping transports that satisfy the constraint will be arbitrarily complex [23]. Third, defining a numerical loss $\mathcal{L}$ cannot simply guarantee functionality preservation.

In this subsection, we focus on addressing the first two challenges. We first replace the similarity measurement $\mathcal{C}(\cdot, \cdot)$

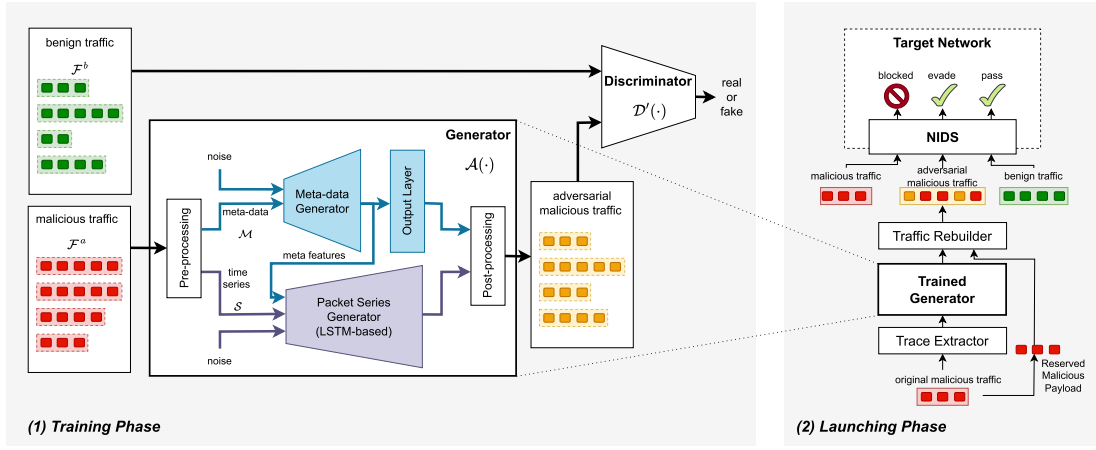


Fig. 3. The ProGen framework using DoppelGANger, which includes a generator and a discriminator. In the (1) Training Phase, the DoppelGANger is trained to generate adversarial traffic traces by taking both the original traffic traces and noise as inputs. The generator consists of a **Meta-data Generator**, which learns from meta-data (traffic identification), and a **Packet Series Generator** to learn the temporal information among the measurements of packets in traffic. To learn the correlations between the meta-data and measurements, the learned meta-features are fed to the Packet Series Generator. The discriminator classifies both the benign traffic trace data and generated adversarial traffic traces for training purposes. In the (2) Launching Phase, original malicious traffic is transformed as trace data for the trained generator, and the malicious payload is reserved. Then, the generator generates adversarial malicious traffic traces. After that, the adversarial malicious traffic is rebuilt from both the adversarial malicious traffic traces and the reserved malicious payload.

in Equation (5) with a comparison of the classification logistics between $\mathcal{D}'(\mathcal{A}(\mathcal{F}^a))$ and $\mathcal{D}'(\mathcal{F}^b)$, where $\mathcal{D}'(\cdot)$ is a surrogate discriminator serving as a proxy for the unavailability of the target model $\mathcal{D}(\cdot)$. Then we transform the optimization problem (5) into a generative learning problem.

In Wasserstein GAN [3], Wasserstein distance, which is a measure of the discrepancy between two probability distributions, is applied in the discriminator's objective function. In our problem, the loss function of WGAN's discriminator is defined as

$$W(\mathcal{P}_s, \mathcal{P}_t) = \max_{\omega'} \mathbb{E}_{\mathcal{F}^b \sim \mathcal{P}_t} (\mathcal{D}'_{\omega'}(\mathcal{F}^b)) - \mathbb{E}_{\mathcal{F}^a \sim \mathcal{P}_s} (\mathcal{D}'_{\omega'}(\mathcal{A}(\mathcal{F}^a))), \quad (6)$$

where ω' is the trainable weights of the discriminator $\mathcal{D}'$. Although the equation (6) is a maximization problem (different from the minimization problem in Equation (5)), we know that the final goal of GAN is fooling the discriminator, which is actually making $\mathbb{E}_{\mathcal{F}^b \sim \mathcal{P}_t} (\mathcal{D}'_{\omega'}(\mathcal{F}^b)) \approx \mathbb{E}_{\mathcal{F}^a \sim \mathcal{P}_s} (\mathcal{D}'_{\omega'}(\mathcal{A}(\mathcal{F}^a)))$. To achieve this goal, the WGAN's generator is trained by minimizing the objective function defined as

$$L = \min_{\theta} \mathbb{E}_{\mathcal{F}^a \sim \mathcal{P}_s} (\mathcal{D}'(\mathcal{A}_{\theta}(\mathcal{F}^a))), \quad (7)$$

where θ' is the trainable weights of the generator $\mathcal{A}$. Therefore, we believe that the Wasserstein GAN can be adopted to solve the projecting-based adversarial malicious traffic generation problem.

C. Traffic Space GAN–DoppelGANge

The DoppelGANger is a GAN model specifically designed for networking tasks. Its primary function is adopted by Netshare [35] to automatically learn and generate synthetic packet traces and flow header traces. Three insight designs make DoppelGANger perfectly suitable for our problem defined in Section IV-B, which are

- Treating the traffic trace data as a time series, considering each packet in the context of flows as the unit;
- Separately processing the meta-data and time series of traffic, but still capturing their correlation;
- Using network domain knowledge to design the input encoding and normalizing methods to improve model performance.

Hence, we utilize the DoppelGANger architecture to train a traffic generator, which serves as the projecting-based adversarial operator denoted by $\mathcal{A}(\cdot)$ within the BFS representation space.

1) *GAN Structure*: The structure of DoppelGANger is shown in the left part of Figure 3. Same as all GAN structures, DoppelGANger includes a generator and a discriminator denoted as $\mathcal{A}(\cdot)$ and $\mathcal{D}'(\cdot)$ respectively.

The generator $\mathcal{A}(\cdot)$, a crucial component, consists of two distinct parts: the Meta-data Generator $\mathcal{A}^m(\cdot)$ and the Packet Series Generator $\mathcal{A}^s(\cdot)$. The Meta-data Generator is built with Multilayer Perceptron (MLP). It focuses on learning from the meta-information $\mathcal{M}$, such as traffic identification, to gain insights into the traffic's characteristics. On the other hand, the Packet Series Generator is built with Long Short-Term Memory (LSTM). It concentrates on understanding the temporal relationships among basic feature sequences $\mathcal{S}$ (*insight design (a)*). To establish the connections between the meta-information and the basic feature sequences, the learned meta-features are passed to the Packet Series Generator (*insight design (b)*). The generator is trained by the objective function in Equation (7), which represents whether the generated adversarial malicious traffic will be classified as benign traffic by the discriminator.

The discriminator $\mathcal{D}'(\cdot)$ plays a role by classifying both the benign traffic and the adversarial traffic generated by the generator. This classification process is essential for training traffic GAN, as it provides valuable feedback on the quality and authenticity of the generated adversarial traffic. By iteratively

TABLE II
THE WORD2VEC PARAMETERS FOR META-INFORMATION EMBEDDING

Elements	Dimension	Scale	Description
$\mathbf{ip}_{src}$	16	[0, 1]	source IP
$\mathbf{ip}_{dst}$	16	[0, 1]	destination IP
$\mathbf{port}_{src}$	10	[0, 1]	source port
$\mathbf{port}_{dst}$	10	[0, 1]	destination port
$\mathbf{proto}$	2	[0, 1]	protocol
$\mathbf{id}$	32	[0, 1]	traffic ID

refining the generator based on the discriminator's feedback, the traffic GAN learns to produce adversarial traffic that closely resembles real benign traffic. At the same time, the discriminator is trained by the objective function in Equation (6), in which the discriminator is optimized to distinguish the real benign traffic and the generated adversarial malicious traffic.

In the training phase, the generator and discriminator are alternately trained on the collected malicious and benign traffic sets pair Ω_s and Ω_t . During the contest between the generator and discriminator, the generator learned an approximate distribution mapping function from the source (malicious) distribution to the target (benign) distribution in the BFS representation space. The generated adversarial traffic will not only fool the trained discriminator $\mathcal{D}'(\cdot)$ but also the targeted NIDS classifier $\mathcal{D}(\cdot)$.

2) *Data Processing*: Given the significant heterogeneity in traffic features as discussed in prior works [18], [22], it is crucial to employ appropriate data pre-processing techniques, accommodating the diverse data types and scales of these features to optimize learning performance. Then post-processing is also essential to recover the generated feature back to the original data type for rebuilding the adversarial traffic.

Meta-information, denoted as $\mathcal{M}$, is a set of traffic-level information. Instead of numerical variables, all elements in $\mathcal{M}$ are close to the symbols for distinguishing each traffic flow from the rest in network transmission. Hence, we adopt the word2vec approach used in the NLP field to encode them into vector embedding and then feed them into the neural networks. Given the elements, which are "destination port", "source IP", "destination IP", "source port", "protocol", and "traffic ID", have different formats and ranges, they must be encoded by customized word2vec models. More details about meta-information encoding are listed in the following table.

The dimensions of embedding in Table II are reduced in comparison to the length of the original binary meta-information elements. For instance, the IP address, which typically comprises 32 bits, is encoded into only 16 dimensions. This reduction is because of the limitations of the collected data, which cannot encompass all potential addresses. Minimizing the dimensions not only for this dimension constraint but also leads to a smaller neural network model requirement.

The sequence of packet-level measurements within the set $\mathcal{S}$ illustrates the behavior of the traffic flow, providing insights into whether the traffic is benign or malicious. Due to the adopted measurement sequences being heterogeneous, we individually process each series with normalization or encoding methods depending on its data type. For example,

TABLE III
THE NORMALIZATION AND ENCODING APPROACH APPLIED FOR THE ELEMENTS OF BFS

Elements	Type	Normalization/Encoding	Description
$\mathbf{t}_i$	float	min-max	timestamp
$\mathbf{d}_i$	categorical	one-hot	packet direction
$\mathbf{h}_i$	categorical	one-hot	header length
$\mathbf{p}_i$	integer	min-max	payload length
$\mathbf{f}_i$	categorical	one-hot	flag
$\mathbf{w}_i$	integer	min-max	TCP windows

the "payload length" is normalized by its min-max scale from 0 to 65535 in units of bytes. Different from the payload, the "header length" consists of different layers, such as IP header, and transport layer header, that have the fixed formats and options. Therefore, treating "header length" as categorical classes, which have limited fixed choices, is more suitable than continuing integers. More details about meta-information encoding are listed in the following table.

Similarly, post-processing is important for recovering and decoding the neural network outputs back to their original scale so as to rebuild legitimate generated traffic. The post-processing is exactly a reversed version of pre-processing.

3) *Traffic Rebuild*: After the generator outputs the adversarial BFS, based on which there is a traffic rebuild module for manipulating the original traffic and mapping them to benign distribution. First, we reserve the malicious content and split them into N_s parts where N_s refers to the packet number from the attacker to the victim. Then, we modify the header length. For the small header length, we split the transport layer header into a few IP packets. For the large header length, we pad zero bits in the options position. For the TCP window size, we directly modify the window size bits in the TCP header. For the flag information, we set the generated flags in the corresponding bits. If some flags are relevant to the malicious functionality, we will not change them. For the payload length, we can pad with extra meaningless content. Finally, we send out the packets with the generated Intervals. Spoof source addresses and source port numbers are also common ways to pass by IDSs undetected. Manipulating the packet direction is hard and special to manipulate. The NIDS and victims are not in the same position; therefore, we also can generate spoofed packets from the victim IP to the attacker

D. Realistic Generation Constraints

Although the DoppelGANger is designed for network data, its original task—generating synthetic traffic trace is different from adversarial traffic generation. We observe the following gap between adversarial and synthetic generation:

- (b) The synthetic traffic normally lasts for a long duration, therefore the traffic GAN needs to learn long-time and cross-flow dependence. However, generating adversarial examples focuses on each flow's behavior independently.
- (b) The synthetic traffic can be assigned to any class based on its behavior. However, adversarial traffic must meet the characteristics of both its source class $\mathcal{C}_s$ and target class $\mathcal{C}_t$.

In order to ensure that the generated adversarial traffic is realistic and effective, we add constraints in both pre-processing and training loss.

Constraint 1: To bridge the gap (a), we employ timestamp shift pre-processing to reduce the wide-range distribution of real-world packet timestamps. The traffic GAN is trained to approximate the global distribution of benign traffic. Based on the experimental results, we have observed that the wide-range timestamps in benign traffic can cause the generated flow duration to be out-of-distribution, particularly excessively large. To address this issue, we adopt a simple timestamp shift pre-processing as Equation (8).

$$t_{i,j} = t_{i,j} - (t_{i,0} - t_{min}) + \frac{i \times D}{N}, \quad (8)$$

where $T_{i,j}$ is the timestamp of the j th packet in traffic flow i . T_{min} is the minimum timestamp of all packets. N is the total number of traffic flows. D is the customized total duration time, which is used to control the generated duration. Equation (8) can move all traffic flows' start timestamps but keep the inter-arrival time of each packet.

Constraint 2: To bridge the gap (b), we extend the original Wasserstein loss [3] in DoppelGANger with a perturbation loss, which consists of two extra weighted mean square error (WMSE) losses for both the meta-data generator and the packet series generator. The perturbation loss calculates the difference between the inputs and outputs of the two generators so that the automatic perturbation learned by generators can be restricted in a reasonable range. The extra joint loss $l_{perturb}$ function is defined as Equation (9).

$$l_{perturb} = a_1 \cdot WMSE(\mathcal{A}^m(\mathcal{M}^a), \mathcal{M}^b) + a_2 \cdot WMSE(\mathcal{A}^s(\mathcal{S}^a), \mathcal{S}^b), \quad (9)$$

where $\mathcal{A}^m(\mathcal{M}^a)$ and $\mathcal{M}^b$ are the generated and original input meta-data embedding, their $WMSE$ loss has a joint coefficient a_1 , and $\mathcal{A}^s(\mathcal{S}^a)$, $\mathcal{S}^b$, and a_2 are used for packet series embedding. The $WMSE$ loss is formulated as:

$$WMSE(\hat{\mathbf{x}}, \mathbf{x}) = \frac{\sum_{i=1}^d \mathbf{w}_i (\hat{\mathbf{x}}_i - \mathbf{x}_i)}{d \times \sum_{i=1}^d \mathbf{x}_i}, \quad (10)$$

where $\hat{\mathbf{x}}, \mathbf{x}, \mathbf{w} \in \mathbb{R}^d$, both $\hat{\mathbf{x}}$ and $\mathbf{x}$ are the concatenated elements of $\mathcal{M}$ or $\mathcal{S}$, based on different i we can assign weights for each element. For example, $\{\mathbf{x}_i^m, i = 43, 44, \dots, 52\}$ corresponds to the binary encoding vector of the destination port number. When learning to generate an adversarial File Transfer Protocol (FTP)-brute force attack, the destination port should not be perturbed, therefore we assign larger weights $\mathbf{w}_i, i = 43, 44, \dots, 52$.

To demonstrate the influence of the above constraints on the fidelity and realism of the generated adversarial attacks, we present the distributions of several key elements of BFS representation of generated adversarial traffic in Sec. V-B.

E. Functionality Preservation

Functionality preservation is critical for the generated adversarial traffic. By training the traffic GAN with realistic constraints while retaining the original malicious payload,

TABLE IV
THE FUNCTIONAL BEHAVIOR ANALYSIS FOR THE
ATTACKS INCLUDED IN OUR EXPERIMENTS

Dataset	Attack type	Attack name	Functional behavior
CIC18	Bruteforce	FTP – Patator	high rate, dport, fixed proto
		SSH – Patator	high rate, dport, fixed proto
	Slow DoS	Slowloris,	low rate, flags, fixed proto
		Slowhttptest	low rate, flags, fixed proto
CIC17	PortScan	Nmap	various port, various proto
UNSW15	Fuzzers	SMB based	dport, fixed proto
		FTP based	dport, fixed proto
		HTTP based	dport, fixed proto
	Analysis	Port Scanner	various port, various proto

we can ensure the preservation of communication functionality and, in turn, contribute to maintaining the malicious function. However, it's still not enough to make sure the adversarial malicious traffic can damage the target network environment.

We believe preserving the functional behavior of adversarial traffic needs to be individually designed based on the network attack type $\mathcal{C}_s$. By analyzing the feature extraction and various attack behaviors in NSL-KDD dataset [16], Usama et al. [30] aim to preserve traffic's functional behavior by keeping the malicious behavior-related features untouched. In this work, we analyze the technical principles of each attack type and assign different weights $\mathbf{w}_i$ in Equation (10) for different attack types. The attack functional behaviors are summarized in Table IV. In our training implementation, we set the default weights $\mathbf{w}_i$ to be 1, where i is the element position in BFS. For the functional behavior-related elements in BFS, we use a large $\mathbf{w}_i = 10$. Then, we need to analyze each type of attack to determine its functional behavior-related elements.

1) Case Study on Functionality Preservation: We show a case study on generating functionality preservation adversarial Slow DoS attack. The Slow DoS attacks in both CSE-CIC-IDS2018 and CIC-IDS-2017 datasets are generated by an attack tool *SlowHttpTest*. The two specific types of Slow DoS attacks are the Slowloris attack and the Slowhttptest attack.

The Slowloris attack's malicious behavior is sending unfinished HTTP requests and the Slowhttptest attack's malicious behavior is sending unfinished HTTP message bodies. Therefore, to preserve the malicious function the generated adversarial attack must maintain the original protocol, port number, and necessary content in the payload, such as "POST", "GET", and the fixed URL of the target server. By using BFS and the traffic GAN, we can fix the destination port and protocol corresponding to elements $\mathbf{port}_{dst}$ and $\mathbf{proto}$ in meta-information $\mathcal{M}$ and the header length $\mathbf{h}_j$ in the feature sequence $\mathcal{F}$. The generated payload length $\mathbf{p}_i$ must be longer than the necessary payload content. The rest of the BFSs have the freedom to be replaced by the values generated by traffic GAN.

2) The Realism Validation: Although many constraints are designed to preserve the realism of adversarial traffic, we notice that generating realistic synthetic network traffic is inherently challenging, due to hard to capture every single inter-packet and intra-packet correlation [38]. Therefore, we further design realism validation rules to verify whether the generated adversarial traffic is realistic. The validation items

TABLE V
THE ML CROSS-VALIDATION ON CIC-IDS-2017 DATASET

Model	Xgboost				KNN				RF			
	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec
1	0.935	0.922	0.977	0.9081	0.870	0.849	0.923	0.824	0.962	0.945	0.958	0.945
2	0.969	0.974	0.967	0.985	0.805	0.726	0.834	0.753	0.996	0.996	0.996	0.996
3	0.988	0.990	0.985	0.995	0.935	0.902	0.918	0.909	0.997	0.998	0.998	0.997
4	0.997	0.998	0.997	0.998	0.894	0.844	0.860	0.853	0.997	0.997	0.998	0.996
5	0.956	0.960	0.949	0.977	0.834	0.784	0.824	0.793	0.980	0.980	0.977	0.984
ave	0.969	0.969	0.975	0.973	0.868	0.821	0.872	0.826	0.986	0.983	0.985	0.984

TABLE VI
THE DL CROSS-VALIDATION ON CIC-IDS-2017 DATASET

Model	CNN				LSTM				FT			
	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec
1	0.794	0.673	0.747	0.710	0.798	0.782	0.809	0.806	0.872	0.855	0.873	0.858
2	0.832	0.785	0.894	0.771	0.881	0.836	0.841	0.856	0.830	0.832	0.852	0.874
3	0.863	0.855	0.869	0.876	0.879	0.874	0.883	0.869	0.931	0.938	0.937	0.946
4	0.942	0.943	0.956	0.934	0.915	0.921	0.926	0.924	0.975	0.977	0.971	0.983
5	0.862	0.853	0.903	0.826	0.857	0.831	0.849	0.831	0.951	0.957	0.957	0.958
ave	0.859	0.822	0.874	0.824	0.866	0.849	0.862	0.857	0.912	0.912	0.918	0.924

TABLE VII
THE ML CROSS-VALIDATION ON CSE-CIC-IDS2018 DATASET

Model	Xgboost				KNN				RF			
	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec
1	0.754	0.733	0.791	0.809	0.747	0.790	0.888	0.744	0.906	0.935	0.969	0.925
2	0.825	0.808	0.953	0.800	0.844	0.852	0.938	0.814	0.909	0.938	0.970	0.930
3	0.907	0.936	0.971	0.928	0.774	0.800	0.822	0.840	0.863	0.912	0.915	0.913
4	0.907	0.936	0.971	0.928	0.767	0.791	0.882	0.804	0.818	0.894	0.894	0.894
5	0.898	0.945	0.947	0.958	0.718	0.696	0.732	0.701	0.877	0.809	0.826	0.915
ave	0.858	0.872	0.927	0.885	0.770	0.786	0.853	0.780	0.875	0.898	0.915	0.915

TABLE VIII
THE DL CROSS-VALIDATION ON CSE-CIC-IDS2018 DATASET

Model	CNN				LSTM				FT			
	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec
1	0.844	0.842	0.859	0.844	0.701	0.605	0.724	0.577	0.842	0.798	0.829	0.807
2	0.828	0.661	0.655	0.796	0.800	0.651	0.738	0.619	0.929	0.959	0.956	0.964
3	0.904	0.767	0.802	0.750	0.799	0.710	0.771	0.687	0.932	0.954	0.951	0.961
4	0.906	0.901	0.931	0.887	0.816	0.711	0.793	0.678	0.918	0.951	0.951	0.954
5	0.841	0.722	0.768	0.818	0.751	0.619	0.743	0.577	0.893	0.833	0.859	0.883
ave	0.8650	0.778	0.803	0.819	0.773	0.659	0.754	0.628	0.903	0.899	0.909	0.914

are designed as follows: (1) Packet Timing, verifying whether the average inter-arrival time and duration value are within the reasonable scale; (2) Traffic Volume, verifying whether the total payload volume is enough for carrying the original malicious content; (3) Transmission Rate, verifying whether the transmission rate conforms the malicious character; (4) Transport Layer Analysis, verifying whether the transport protocol and source/destination ports are the same as the malicious traffic; (5) Application Layer Analysis, verifying whether the application protocol is the same as the malicious traffic; (6) Flags, verifying whether the flags sequence in conversation and whether the flags are set as the malicious functions.

V. EXPERIMENT AND RESULTS

In this paper, we aim to design a deep generative model-based adversarial attack method against NIDS scenarios. Our method is designed by taking practical networking and security circumstances into account to generate realistic adversarial traffic which remains their communication and malicious functions. Therefore, our evaluation not only considers the evasion effectiveness but also considers whether the generated adversarial traffic fails to be realistic or not.

In this section, we first introduce the experimental settings in Sec. V-A. Then we analyze the realism of the projecting-based adversarial generation by showcasing the generated distributions in Sec. V-B. Finally, we use ProGen to

TABLE IX
THE ML CROSS-VALIDATION ON UNSW-NB15 DATASET

Model	Xgboost				KNN				RF			
	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec
1	0.882	0.890	0.878	0.905	0.812	0.695	0.785	0.681	0.880	0.887	0.878	0.898
2	0.923	0.918	0.938	0.905	0.828	0.706	0.722	0.697	0.922	0.918	0.941	0.904
3	0.948	0.934	0.927	0.946	0.841	0.711	0.731	0.704	0.945	0.941	0.941	0.943
4	0.972	0.969	0.972	0.967	0.862	0.745	0.839	0.725	0.971	0.966	0.961	0.972
5	0.915	0.852	0.834	0.922	0.808	0.684	0.715	0.676	0.908	0.876	0.852	0.932
ave	0.928	0.913	0.910	0.929	0.830	0.708	0.758	0.697	0.925	0.918	0.914	0.930

TABLE X
THE DL CROSS-VALIDATION ON UNSW-NB15 DATASET

Model	CNN				LSTM				FT			
	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec	Acc	F1	Prec	Rec
1	0.871	0.829	0.827	0.832	0.757	0.763	0.774	0.768	0.771	0.771	0.784	0.770
2	0.866	0.811	0.827	0.803	0.776	0.775	0.795	0.777	0.779	0.773	0.779	0.776
3	0.803	0.746	0.750	0.744	0.735	0.739	0.761	0.738	0.804	0.797	0.816	0.802
4	0.855	0.827	0.836	0.819	0.827	0.829	0.841	0.827	0.882	0.874	0.878	0.872
5	0.853	0.788	0.782	0.804	0.727	0.720	0.752	0.725	0.776	0.763	0.781	0.773
ave	0.849	0.801	0.804	0.801	0.765	0.765	0.785	0.767	0.802	0.796	0.808	0.799

evaluate six common ML models to show the adversarial effect in Sec. V-C.

A. Experiment Setup

Our experiments use 3 datasets and 6 NIDS models to demonstrate the realism of generated adversarial traffic and evaluate the effectiveness of adversarial attacks. For the realism aspect, we demonstrate it by showing the distribution of the generated BFS with different constraints in Section V-B. For the effectiveness aspect, we evaluate it by showing the NIDS detection performance decline under the generated attacks.

1) *Dataset*: In this paper, we utilize the tabular data and raw Pcaps of 3 datasets (UNSW-NB15 [36], CIC-IDS-2017 (corrected) [27], [37], and CSE-CIC-IDS2018 [27]) to evaluate the proposed ProGen method. For training and testing the detection baseline of target NIDS models, we use all attack classes except the classes that have insufficient samples. The excluded classes include the infiltration, and web attacks in CIC-IDS-2017 and CSE-CIC-IDS2018, and backdoor attack and worm attacks in UNSW-NB15. For performing adversarial attacks generated by ProGen, we select 5 types of attacks including Bruteforce, Slow DoS, PortScan, Fuzzers, and Analysis attacks from the datasets based on many facts, such as (1) if the sample number is sufficient for training both the ProGen model and targeted NIDS models; (2) if the attack types need to evade detection.

2) *Target NIDS Models*: To evaluate ProGen, we train both shallow ML-based NIDS models and DL-based NIDS models on 3 datasets respectively. For the shallow models, we utilize K-Nearest Neighbor (KNN), Random Forest (RF), XGboost, implemented using sklearn and XGBoost libraries. For the deep models, we employ Convolutional Neural Networks (CNN), LSTM, and Transformer (TF), implemented using PyTorch. We adopt 5-fold cross-validation to evaluate the target NIDS models. On each dataset, each targeted model is trained on 20,000 benign data samples and 20,000 mixed malicious samples and is tested on 5000 benign data samples and 5000 mixed malicious samples. Their multiple-class classification performances on 3 datasets are shown in Table V-VIII.

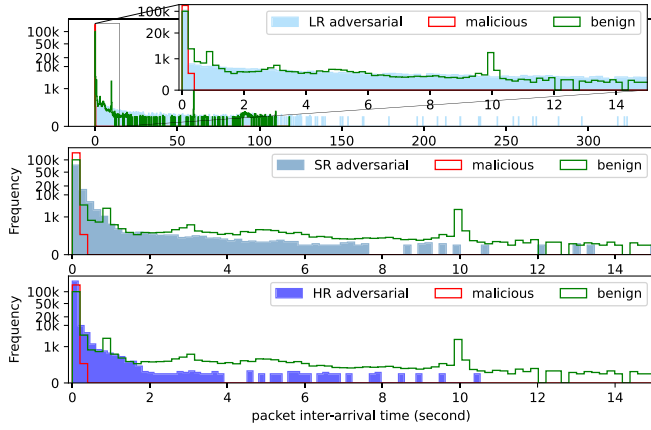


Fig. 4. The packet inter-arrival time distributions for “benign”, “malicious”, and “adversarial” types of traffic in the Slow DoS experiment.

3) *Evaluation Metrics*: In addition to the typical metrics, such as Accuracy, F1-score Precision, and Recall, we define other metrics for evaluating the adversarial attack’s evasion effectiveness and realism. To evaluate the evasion effectiveness, we define the **S (success) number**, **F (fail) number**, and **A (ambiguous) number**, where the S number refers to the number of malicious adversarial traffic classified as benign traffic, the F number refers to the number of malicious adversarial traffic classified as its original attack class, and the A number refers to the number of malicious adversarial traffic classified as its other attack class. To evaluate the realism, we define **PR** (Pass Ratio), the ratio of the generated traffic that can pass the realism validation in Sec. IV-E.

B. The Realism of Adversarial Traffic

To analyze the fidelity and realism of the generated adversarial malicious traffic, we present the distributions of several key elements of BFS representation for Slow DoS attacks introduced in Sec. IV-E. Different from the flooding DoS attack, Slow DoS attacks exhaust the service resource with low-speed unfinished requests and keep the service connections live.

By comparing the distributions generated with different constraint options (Sec. IV-D), the results demonstrate the impact of the constraints on the traffic GAN in generating adversarial attacks. In our experiments, we define the models with different realistic levels based on the training constraints as follows:

- LR: lightly realistic (LR) using no constraints.
- SR: semi-realistic (SR) using constraint 1 only.
- HR: highly realistic (HR) using both constraints 1 and 2.

To demonstrate the effect of the **timestamp shift pre-processing** (constraint 1), we present the generated distributions of temporal information. Figure 4 and 5 illustrate the packet inter-arrival time distribution and flow duration distribution, respectively. The duration upper boundary of both malicious and benign traffic is 120 seconds, as determined by the data processing tool. But the actual maximum duration of Slow DoS attacks is 11.2 seconds. We zoom the x-axis of Figure 4 in the range between 0 to 15 seconds to better show

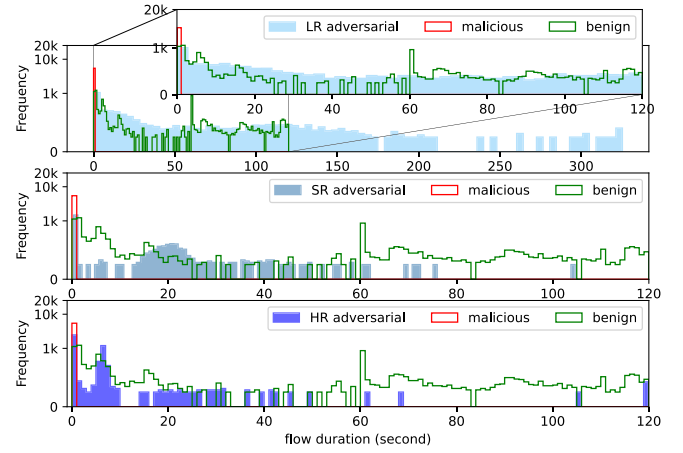


Fig. 5. The flow duration distributions for “benign”, “malicious”, and “adversarial” types of traffic in the Slow DoS experiment.

the difference among adversarial distributions generated with different constraints. In Figure 4, the unconstrained adversarial packet inter-arrival time distribution (top sub-figure) exhibits a wide range from 0 to 330 seconds, which is more than twice that of both the malicious and benign distributions. However, when applying constraint 1, the generated adversarial packet inter-arrival time distributions (the middle and bottom sub-figures) exhibit better realism compared to the unconstrained distribution.

Similarly, Figure 4 also illustrates the impact of constraint 1 on the generated flow duration distributions. The unconstrained adversarial flow duration distribution is notably distant from both the malicious and benign distributions. However, by applying constraint 1, the generated adversarial flow duration distribution shifts towards the region of overlap with the malicious and benign distributions. Furthermore, in our experiments, the target NIDSs have a flow timeout threshold of 120 seconds, which means that unconstrained adversarial Slow DoS traffic with a duration longer than 120 seconds cannot be considered a valid input for the target NIDS model. Therefore, constraint 1 plays a vital role in enabling traffic space GAN to generate realistic timestamps for adversarial traffic attacks.

The cover ranges of SR and HR adversarial distributions with constraint 1 and constraint 1&2 (the sub-figures at the middle and bottom) do not exhibit significant differences in both Figure 4 and 5. However, the peaks of distribution histograms show that the HR distributions can better overlap with the peaks of benign and malicious distributions. For example, in the bottom sub-figures of both Figures 4 and 5, the HR inter-arrive and duration distribution peaks around zero better match the pattern of malicious distribution. In addition, the HR duration distribution peak better imitates the benign distribution than LR and SR distributions in Figure 5.

Furthermore, we illustrate the impact of the **perturbation loss** (constraint 2) by comparing the distributions of packet payload length and the number of packets per flow in Figure 6 and 7. The purpose of constraint 2 is to reserve the malicious function-relevant features of the generated adversarial traffic. In Figure 6, we observe that the payload length distributions of

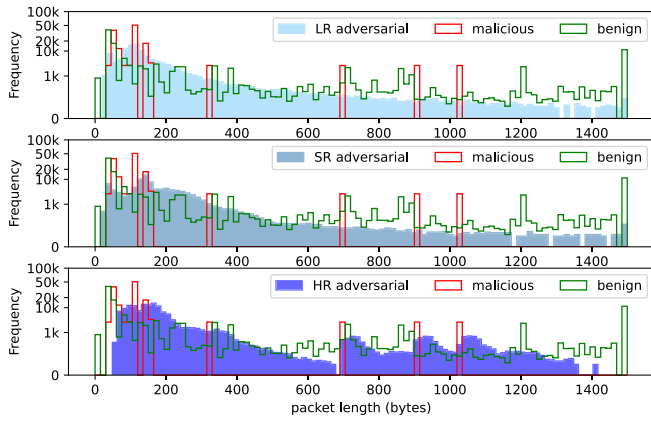


Fig. 6. The packet payload length distributions for “benign”, “malicious”, and “adversarial” types of traffic in the Slow DoS experiment.

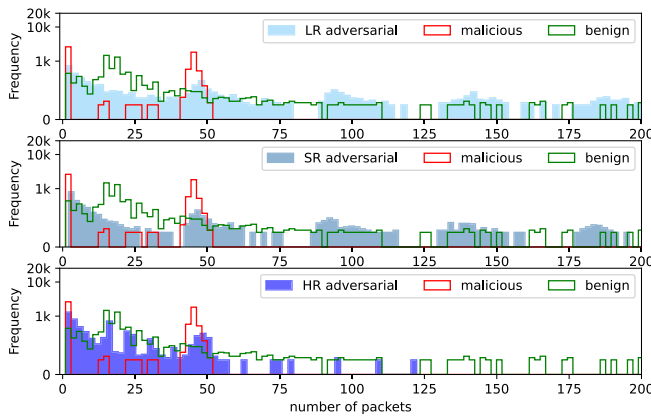


Fig. 7. The number of packets per flow distributions for “benign”, “malicious”, and “adversarial” types of traffic in the Slow DoS experiment.

malicious and benign traffic exhibit notable differences. The malicious payload length distribution is more sparse than the benign one because the malicious traffic is generated by the attack tool with pre-defined header and body length parameters in the command line. Compared with the LR and SR distributions in Figure 6, which is dense and smooth, the HR distribution has several local peaks that match the malicious distributions. This phenomenon indicates the significant role of constraint 2 in reserving the features of malicious traffic.

In Figure 7, it can be observed that without the perturbation loss (constraint 2), the generated adversarial traffic exhibits a higher number of packets per flow. While this generated distribution may resemble the distribution of benign traffic, it is important to note that if the generated flow duration distribution is as short as shown in Figure 5, the large number of packets per flow can result in a high packet arrival rate, potentially triggering alarms in NIDSs.

C. The Effectiveness of Adversarial Traffic

To comprehensively analyze the effectiveness of the adversarial malicious traffic generated by ProGen, we evaluate the 6 trained target NIDS models with generated adversarial

Slow DoS, Brute Force, Port Scan, Fuzzers, and Analysis attacks. Adversarial traffic aims to evade the detection of NIDS, therefore the model’s detection performance decline can reflect the adversarial effectiveness. Besides the effectiveness of generated traffic, we are also interested in the effectiveness of the proposed realistic generation constraints.

Table XI reports the intrusion detection performance of 6 target models in terms of Precision, Recall, and F1-score. For each target model and each class of attack, we test them by 4 different types of adversarial traffic generated by ProGen with different realistic (None, LR, SR, and HR) levels which are also used in Sec V-B. By comparing the intrusion detection model detection performance between adversarial-free and three adversarial traffic with varying realistic levels (LR, SR, HR), we observe the following aspects of information.

1) *Adversarial Type Trends*: Across various models and attacks, when no adversarial traffic (None) is introduced, the NIDS models achieve high precision, recall, and F1 scores for most attack types and datasets. This establishes a baseline for the NIDS models’ performance in the absence of adversarial input. Normally, the adversarial traffic (LR, SR, and HR) will cause a notable F1 decrease across models and datasets, which means ProGen can project the malicious traffic out of distribution. Furthermore, SR’s F1 scores generally show better than the LR, indicating that constraint 1 restricts the adversarial changing scale. The HR case (constrained by Constraints 1 and 2) shows even better evasion effectiveness across most models and attacks than LR and SR. In our design, the constraints are beneficial for the realism of generation. However, the results show that the constraints also improve the evasion effectiveness. To figure out the reason behind this, we analyze the adversarial effectiveness in Table XII. For the extreme metric values, the precision is 1, indicating that no benign traffic is classified as malicious. The precision and recall are equal to 0 means that all malicious traffic fails to be correctly classified.

2) *Model and Dataset-Specific Trends*: Different models exhibit varying degrees of sensitivity to adversarial traffic, with some models being more robust than others. The XGBoost and RF generally demonstrate higher performance across different scenarios, while KNN, CNN, LSTM, and Transformer models may show more sensitivity to adversarial inputs. We also noticed that LSTM has a relevant lower baseline performance. All models exhibit good performance on CIC17 and CIC18 datasets in detecting adversarial-free traffic but struggle to effectively detect adversarial traffic. The UNSW15 shows challenges to DI models (CNN, LSTM, and TF).

Table XII reports more detailed results corresponding the Table XI. In Table XII, the S (success), F (fail), and A (ambiguous) numbers are used to evaluate the effectiveness of adversarial attacks, and the Pass Ratio (PR) is used to evaluate the realism of the generated adversarial traffic. In general, there is a trend of increasing Pass Ratio from LR to HR. This trend can prove that the designed constraints can improve the realism of generated adversarial attacks.

In most existing work, only the success and failure numbers of adversarial attacks are taken into account. However, the

TABLE XI

NIDS MODEL DETECTION PERFORMANCE (INCLUDES PRECISION, RECALL, AND F1-SCORE) COMPARISON BETWEEN ADVERSARIAL-FREE AND THREE ADVERSARIAL TRAFFIC WITH VARYING REALISTIC LEVELS: LIGHTLY-REALISTIC (LR) USING NON CONSTRAINTS, SEMI-REALISTIC (SR) USING CONSTRAINT 1 ONLY AND HIGHLY-REALISTIC (HR) USING BOTH CONSTRAINTS 1 AND 2

Models	Adv Type	CIC18 (Slow DoS)			CIC18 (Brute Force)			CIC17 (PortScan)			UNSW15 (Fuzzers)			UNSW15 (Analysis)		
		Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1
XGB	None	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.999	1.000	0.999	1.000	1.000	1.000
	LR	1.000	0.613	0.747	1.000	0.101	0.181	1.000	0.896	0.945	0.993	0.201	0.334	0.000	0.000	0.000
	SR	1.000	0.562	0.717	1.000	0.164	0.281	1.000	0.472	0.641	0.962	0.026	0.050	1.000	0.010	0.019
	HR	1.000	0.592	0.740	1.000	0.734	0.844	0.000	0.000	0.000	0.979	0.049	0.093	0.000	0.000	0.000
KNN	None	0.999	1.000	1.000	0.999	0.998	0.998	1.000	1.000	1.000	0.966	0.908	0.936	0.971	0.938	0.954
	LR	0.957	0.023	0.045	0.000	0.000	0.000	1.000	0.016	0.031	0.389	0.022	0.041	0.849	0.216	0.344
	SR	0.979	0.050	0.095	0.000	0.000	0.000	1.000	0.004	0.008	0.389	0.078	0.130	0.892	0.409	0.561
	HR	0.981	0.058	0.109	0.000	0.000	0.000	0.000	0.000	0.000	0.083	0.014	0.025	0.913	0.493	0.640
RF	None	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
	LR	1.000	0.119	0.210	0.000	0.000	0.000	1.000	0.649	0.785	1.000	0.492	0.656	0.800	0.012	0.024
	SR	1.000	0.026	0.051	0.000	0.000	0.000	1.000	0.307	0.431	1.000	0.254	0.373	1.000	0.097	0.174
	HR	0.600	0.001	0.003	0.000	0.000	0.000	0.800	0.055	0.100	1.000	0.274	0.417	0.400	0.003	0.006
CNN	None	0.988	0.962	0.974	0.998	1.000	0.999	1.000	0.998	0.999	0.852	0.933	0.891	0.967	0.895	0.928
	LR	0.721	0.274	0.349	0.200	0.115	0.146	0.998	0.084	0.149	0.314	0.180	0.227	0.745	0.538	0.554
	SR	0.660	0.260	0.326	0.845	0.067	0.119	0.990	0.103	0.173	0.285	0.209	0.200	0.833	0.538	0.574
	HR	0.660	0.243	0.309	0.567	0.008	0.016	0.200	0.004	0.007	0.138	0.178	0.144	0.772	0.496	0.534
LSTM	None	0.912	0.678	0.734	0.978	0.951	0.963	1.000	0.997	0.998	0.796	0.817	0.805	0.891	0.728	0.800
	LR	0.073	0.002	0.004	0.408	0.036	0.066	1.000	0.003	0.006	0.360	0.283	0.274	0.852	0.564	0.652
	SR	0.122	0.008	0.014	0.173	0.004	0.008	1.000	0.005	0.011	0.415	0.377	0.376	0.803	0.498	0.593
	HR	0.109	0.006	0.008	0.181	0.008	0.016	0.000	0.000	0.000	0.306	0.401	0.288	0.756	0.437	0.517
TF	None	0.984	1.000	0.992	1.000	1.000	1.000	1.000	1.000	1.000	0.899	0.978	0.937	0.998	0.996	0.997
	LR	0.745	0.055	0.101	1.000	0.057	0.107	1.000	0.238	0.384	0.476	0.245	0.312	0.855	0.296	0.365
	SR	0.354	0.013	0.025	1.000	0.055	0.104	1.000	0.233	0.377	0.502	0.303	0.353	0.944	0.312	0.416
	HR	0.353	0.014	0.028	1.000	0.043	0.080	1.000	0.142	0.205	0.366	0.194	0.235	0.968	0.275	0.399

TABLE XII

ADVERSARIAL EFFECTIVENESS EVALUATION (INCLUDING S/F/A NUMBER AND PR) COMPARISON AMONG VARYING REALISTIC LEVEL ADVERSARIAL TRAFFIC: LIGHTLY-REALISTIC (LR) USING NON CONSTRAINTS, SEMI-REALISTIC (SR) USING CONSTRAINT 1 ONLY, AND HIGHLY-REALISTIC (HR) USING BOTH CONSTRAINTS 1 AND 2

Model	Adv Type	CIC18 (Slow DoS)		CIC18 (Brute Force)		CIC17 (PortScan)		UNSW15 (Fuzzers)		UNSW15 (Analysis)	
		S/F/A	PR	S/F/A	PR	S/F/A	PR	S/F/A	PR	S/F/A	PR
XGB	None	0/1000/0	1.00	0/1000/0	1.00	0/1000/0	1.00	0/923/77	1.00	0/960/40	1.00
	LR	48/412/540	0.37	804/89.8/106.2	0.38	48/412/540	0.37	574/144/282	0.51	837/0/163	0.52
	SR	28/25/947	0.71	703.8/138.6/157.6	0.72	28/25/947	0.71	954/25/21	0.65	625/6/369	0.61
	HR	2/0/998	0.81	262.4/724.4/13.2	0.85	2/0/998	0.81	918/47/35	0.76	830/0/170	0.85
KNN	None	0/999/1	1.00	2/998/0	1.00	0/999/1	1.00	64/633/303	1.00	58/871/71	1.00
	LR	506/8/486	0.37	985/0/15	0.38	506/8/486	0.37	631/14/355	0.51	531/146/323	0.52
	SR	985/4/11	0.71	966/0/34	0.72	985/4/11	0.71	166/14/820	0.65	311/215/474	0.61
	HR	963/0/37	0.81	551/0/449	0.85	963/0/37	0.81	137/2/861	0.76	282/274/444	0.85
RF	None	0/1000/0	1.00	0/1000/0	1.00	0/1000/0	1.00	0/898.4/101.6	1.00	0/961.2/38.8	1.00
	LR	265.4/494.6/240	0.37	928.4/0/71.6	0.38	265.4/494.6/240	0.37	390.4/385.8/223.8	0.51	373/3.2/623.8	0.52
	SR	283/170/547	0.71	1000/0/0	0.72	283/170/547	0.71	280.6/141.2/578.2	0.65	128.6/13/858.4	0.61
	HR	360.2/21.8/618	0.81	1000/0/0	0.85	360.2/21.8/618	0.81	245.4/117.4/637.2	0.76	280.6/0.8/718.6	0.85
CNN	None	2.4/997/0.6	1.00	0/911.6/88.4	1.00	2.4/997/0.6	1.00	29.4/412.6/558	1.00	64.6/545.2/390.2	1.00
	LR	666.6/64.2/269.2	0.37	443.6/75.2/481.2	0.38	666.6/64.2/269.2	0.37	309.8/82.8/607.4	0.51	372.4/346.4/281.2	0.52
	SR	623/73.4/303.6	0.71	437.4/28.8/533.8	0.72	623/73.4/303.6	0.71	409.2/47.2/543.6	0.65	311.8/334.8/353.4	0.61
	HR	968.6/3.8/27.6	0.81	253.8/1/745.2	0.85	968.6/3.8/27.6	0.81	489.8/26.4/483.8	0.76	330.8/340.2/329	0.85
LSTM	None	3/991.6/5.4	1.00	24.6/588/387.4	1.00	3/991.6/5.4	1.00	83/383.2/533.8	1.00	155/413.4/431.6	1.00
	LR	689.2/2/308.8	0.37	579.4/23.4/397.2	0.38	689.2/2/308.8	0.37	307.4/89.8/602.8	0.51	281.2/336.2/382.6	0.52
	SR	860.8/4.6/134.6	0.71	686.6/2.8/310.6	0.72	860.8/4.6/134.6	0.71	291.4/90/618.6	0.65	298.8/254/447.2	0.61
	HR	967.6/0/32.4	0.81	507/2.4/490.6	0.85	967.6/0/32.4	0.81	321.2/58.2/620.6	0.76	339.4/193.2/467.4	0.85
TF	None	0/1000/0	1.00	0/912.6/87.4	1.00	0/1000/0	1.00	10.2/456.8/533	1.00	2.4/627.4/370.2	1.00
	LR	739.4/231.2/29.4	0.37	881/52.6/66.4	0.38	739.4/231.2/29.4	0.37	212.8/58.8/728.4	0.51	155.4/93.8/750.8	0.52
	SR	740.4/225.4/34.2	0.71	899.4/52/48.6	0.72	740.4/225.4/34.2	0.71	210.8/65.6/723.6	0.65	189.4/103.2/707.4	0.61
	HR	855.8/142.4/1.8	0.81	949.6/42.4/8	0.85	855.8/142.4/1.8	0.81	243/38.8/718.2	0.76	205.2/96.8/698	0.85

more practical scenario is that the NIDS models are trained for multi-class classification. Therefore, we also consider the ambiguous number, which indicates that adversarial traffic is classified as another attack class. In this case, even though the results in Table XI show a performance decline, the A number can help us to determine whether the adversarial attacks are really successful or not. In most cases, the increasing success numbers and decreasing fail numbers as the realism constraints become more stringent.

However, the trends are not very clear across all conditions. Different ML and DL NIDS models show variations in their performance under different realism levels and attack types. Some models may be more robust or vulnerable to adversarial attacks, depending on the context. Each model exhibits specific patterns under different settings. For example, some classifiers may perform better under HR conditions, while others may show more consistent performance across LR, SR, and HR.

TABLE XIII
GAN-BASED SYNTHETIC TRAFFIC GENERATION SUMMARIZING

	SynGAN [4]	Flow-WGAN [13]	PacketCGAN [32]	DoppelGANger [19]
Application Scenario	NIDS	NIDS	Encrypted Traffic Classification	Network Traffic Analysis
Generate Data Format	Features	Raw Bytes	Raw Bytes	Time Series
Core DL Module	MLP	GRU, Attention	MLP	LSTM
Evaluation Dataset	NSL-KDD, CICIDS2017	ISCX-2012, ISCX-2017	ISCX2012, USTC-TFC2016	WWT, MBA, GCUT

VI. RELATED WORK

In this section, we introduce the related work and compare existing studies with the proposed ProGen. The primary focus of the related work is on two aspects: GAN-based traffic generation and traffic-space adversarial attack against NIDSs. They are particularly relevant to our chosen techniques and the objectives of our work.

A. GAN-Based Synthetic Traffic Generation

The recent studies on GAN-based synthetic network traffic generation inspire us to design a traffic space adversarial attack against ML-based NIDSs. GAN [10] is a type of DL model that is widely used for generating realistic and high-quality synthetic data, such as images, videos, and even text. GANs have also been applied to generate adversarial attacks in the network domain. For example, SynGAN [4] generates malicious flow feature mutations to evade NIDS detection. Rigaki and Garcia [25] proposed a GAN-based adversarial malware communication to avoid detection.

Recently, many GAN-based synthetic network traffic generation methods are proposed. For instance, Flow-WGAN [13] uses the Wasserstein GAN model [3] to generate new network traffic data from the original data sets to enhance the network packet data. PacketCGAN [32] uses Conditional GAN [34] to generate traffic samples to solve the class imbalance issue. However, most of those approaches utilize existing GAN models, which are designed for natural language or tabular data, and fail to capture the temporal pattern in each flow. A state-of-the-art method Netshare [35] is proposed for generating synthetic packet/flow header traces. Netshare adopts DoppelGANger [19] model, which is particularly designed for traffic trace data. Therefore, Netshare can generate more fine-grained trace records, such as timestamps and header size.

B. Traffic-Space Adversarial Attacks in NIDSs

Recently, many traffic space adversarial attack studies have been proposed [12], [26], [28], [29], [33]. Those works consider realistic NIDS scenarios with a more practical threat model than feature space approaches. With various designs of attack generation, existing traffic space adversarial attack methods claimed multiple technical merits they own. We summarize those technical merits as follows:

- **Black-Box:** Does not require any knowledge about the target NIDS model or its feature set for training.
- **One-Step Optimization:** The adopted ML methods, such as the LSTM model or reinforcement learning method, are one-step trained to generate adversarial traffic.
- **End-to-End Launching:** During the launching phase, the trained model can end-to-end generate adversarial traffic.

TABLE XIV

TECHNICAL MERITS FOR REALISTIC ADVERSARIAL ATTACK METHODS, AND THEIR USAGE IN EXISTING SOLUTIONS AND OUR PROGEN

Technical Merits	Traffic Space Solutions					
	[26]	[12]	[33]	[29]	[28]	ProGen
Black-Box					✓	✓
One-step Optimization	✓		✓	✓	✓	✓
End-to-End Launching			✓	✓	✓	✓
Raw Traffic Info Learning					✓	✓
Automatic Scope Perturbations	✓	✓	✓	✓		✓
Automatic Scale Perturbations					✓	✓
Multi-class NIDS Evaluation						✓

- **Automatic Scope Perturbation** The learning-based capability to automatically perturb various scope of traffic parameter settings.
- **Automatic Scale Perturbation** The learning-based capability to automatically perturb each traffic parameter in a varying scale.

Besides the technical merits, we also consider the challenges: (1) raw traffic data cannot be directly input into ML models; (2) determining appropriate perturbation scale and direction becomes especially challenging, particularly in the case of multi-class NIDS in Table XIV.

Sadeghzadeh et al. [26] proposed Adversarial Network Traffic (ANT) generates adversarial perturbation in three aspects of traffic space packet payload length, packet number, and flow bursts. However, ANT requires full knowledge of the target detection model and the feature set, and different perturb operations are learned separately. Han et al. [12] proposed a two-step solution to practically generate traffic space adversarial attacks against realistic scenarios. They first generate adversarial features with a GAN to let the malicious traffic mimic benign traffic in the feature space. Then, a particle swarm optimization (PSO) is adopted to project the feature perturbation back to the traffic space. Clearly, the two-step method incurs additional costs compared to the one-step approach, and it also requires domain knowledge to guess the feature set for training the feature space GAN.

Both Wu et al. [33] and Tan et al. [29] proposed RL-based evasion attacks against NIDS models. However, RL-based methods require inspecting the feedback of target NIDS to train the RL models. Once their queries are blocked, they cannot finish training the adversarial RL models. An advantage of the attack in [29] is that their framework work can perturb live network traffic, which makes their attack more

TABLE XV
CASE STUDY RESULT: THE CORRELATION BETWEEN BFS ELEMENTS AND THE EXTRACTED FEATURES IN CSE-CIC-IDS2018

Feature Context	Bi-direction			Forward Direction			Backward Direction		
	Num	Feature name	BF	Num	Feature name	BF	Num	Feature name	BF
Packet Size	38	pkt_len_min	p_i	2	tot_fw_pkt	d_i	3	tot_bw_pkt	d_i
	39	pkt_len_max	p_i	6	fw_pkt_l_max	p_i, d_i	10	bw_pkt_l_max	p_i, d_i
	40	pkt_len_avg	p_i	7	fw_pkt_l_min	p_i, d_i	11	bw_pkt_l_min	p_i, d_i
	41	pkt_len_std	p_i	8	fw_pkt_l_avg	p_i, d_i	12	bw_pkt_l_avg	p_i, d_i
	42	pkt_len_va	p_i	9	fw_pkt_l_std	p_i, d_i	13	bw_pkt_l_std	p_i, d_i
	52	pkt_size_avg	p_i	53	fw_seg_avg	p_i, d_i	54	bw_seg_avg	p_i, d_i
	51	down_up_ratio	p_i, d_i	4	tot_fw_pkt_l	p_i, d_i	5	tot_bw_pkt_l	p_i, d_i
Header Size				34	fw_hdr_len	h_i, d_i	35	bw_hdr_len	h_i, d_i
Inter-arrival time	14	fl_byt_s	p_i, t_i						
	15	fl_pkt_s	t_i	36	fw_pkt_s	d_i, t_i	37	bw_pkt_s	d_i, t_i
	1	fl_dur	t_i	20	fw_iat_tot	d_i, t_i	25	bw_iat_tot	d_i, t_i
	16	fl_iat_avg	t_i	21	fw_iat_avg	d_i, t_i	26	bw_iat_avg	d_i, t_i
	17	fl_iat_std	t_i	22	fw_iat_std	d_i, t_i	27	bw_iat_std	d_i, t_i
	18	fl_iat_max	t_i	23	fw_iat_max	d_i, t_i	28	bw_iat_max	d_i, t_i
Flags	19	fl_iat_min	t_i	24	fw_iat_min	d_i, t_i	29	bw_iat_min	d_i, t_i
	43	fin_cnt	f_i						
	44	syn_cnt	f_i						
	45	rst_cnt	f_i						
	46	psh_cnt	f_i	30	fw_psh_flag	d_i, f_i	31	bw_psh_flag	d_i, f_i
	47	ack_cnt	f_i	67	fw_act_pkt	d_i, f_i			
	48	urg_cnt	f_i	32	fw_urg_flag	d_i, f_i	33	bw_urg_flag	d_i, f_i
TCP Window	49	cwe_cnt	f_i						
	50	ece_cnt	f_i						
Bulk and Sub-flow				65	fw_win_byt	d_i, w_i	66	bw_win_byt	d_i, w_i
				55	fw_byt_blk_avg	p_i, d_i, t_i	58	bw_byt_blk_avg	p_i, d_i, t_i
				56	fw_pkt_blk_avg	p_i, d_i, t_i	59	bw_pkt_blk_avg	p_i, d_i, t_i
				57	fw_blk_rate_avg	p_i, d_i, t_i	60	bw_blk_rate_avg	p_i, d_i, t_i
				61	subfl_fw_pkt	p_i, d_i, t_i	63	subfl_bw_pkt	p_i, d_i, t_i
Activate and Idle				62	subfl_fw_byt	p_i, d_i, t_i	64	subfl_bw_byt	p_i, d_i, t_i
	69	atv_avg	d_i, t_i						
	70	atv_std	d_i, t_i						
	71	atv_max	d_i, t_i						
	72	atv_min	d_i, t_i						
	73	idl_avg	d_i, t_i						
	74	idl_std	d_i, t_i						
	75	idl_max	d_i, t_i						
	76	idl_min	d_i, t_i						

practical in the real world. Similarly, Sharon et al. [28] proposed the TANTRA, which can end-to-end execute adversarial attacks by reshaping the original malicious traffic in the time domain. TANTRA trains an LSTM model to learn the temporal behavior of benign traffic within the inter-arrival time prediction task. The trained model is then used to generate new inter-arrival times for malicious traffic. TANTRA does not require any target model or feature set information, but it only perturbs the inter-arrival time. Another shortcoming is that the LSTM model has fixed outputs for specific inputs, which may result in the adversarial attacks having some pattern, which triggers other defense alarms.

The comparison with the existing works in terms of mentioned technical merits is shown in Table XIV. Because of representing traffic with BFS, ProGen doesn't require any feature knowledge. As shown in Table XV, all features will be impacted by BFS. Instead of aiming to cross the decision boundary of a particular model, ProGen learns traffic distribution projection to mimic benign network traffic. So far, knowing about the target NIDS model is not required. In ProGen, traffic space GAN is trained to generate adversarial malicious traffic on collected data with one-step optimization. Therefore, the trained generator can be directly applied end-to-end for

launching the attack. Based on the collected malicious and benign traffic, the traffic GAN will automatically learn how to adjust the BFS elements' scope and scale so that it generates adversarial traffic which behaves like benign traffic.

VII. CONCLUSION

In this study, we introduce ProGen, a projection-based adversarial attack framework designed to produce realistic adversarial network traffic. In ProGen, we introduce the concept of BFS space for representing network traffic that aligns with a realistic threat model. ProGen utilizes a traffic space GAN to approximate the mapping between malicious and benign traffic distributions. To enhance the applicability of the generative model in adversarial attack scenarios, we have also devised constraints aimed at preserving the functionality of the generated adversarial traffic. We assess the effectiveness of ProGen across six common ML models using the CSE-CIC-IDS2018, CIC-IDS-2017, and UNSW-NB15 datasets. We provide visualizations of the generated BFS element distributions to demonstrate the impact of our realistic constraints on the projection process for generating adversarial traffic. Our results reveal that the proposed approach significantly harms detection performance across different ML models.

Generating realistic adversarial attacks against real-world NIDSs requires both theoretical analyses of how to manipulate original malicious traffic to avoid detection and the execution of these manipulations on live traffic. Our work focuses on the theoretical method for generating adversarial traffic in BFS space, and the evaluation relies on offline datasets. The theoretical method we have provided for manipulating original malicious traffic forms the foundation for online validation. In our future work, we will study the impact of real-world network environment factors on the performance of adversarial attacks.

REFERENCES

- [1] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni, "Modeling realistic adversarial attacks against network intrusion detection systems," *Digit. Threats: Res. Pract.*, vol. 3, no. 3, pp. 1–19, 2022.
- [2] G. Apruzzese et al., "The role of machine learning in cybersecurity," *Digit. Threats: Res. Pract.*, vol. 4, no. 1, pp. 1–38, 2023.
- [3] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 214–223.
- [4] J. Charlier, A. Singh, G. Ormazabal, R. State, and H. Schulzrinne, "SynGAN: Towards generating synthetic network attacks using GANs," 2019, *arXiv:1908.09899*.
- [5] A. Chernikova and A. Oprea, "FENCE: Feasible evasion attacks on neural networks in constrained environments," *ACM Trans. Privacy Secur.*, vol. 25, no. 4, pp. 1–34, 2022.
- [6] J. Clements, Y. Yang, A. A. Sharma, H. Hu, and Y. Lao, "Rallying adversarial techniques against deep learning for network security," in *Proc. IEEE Symp. Comput. Intell. (SSCI)*, Dec. 2021, pp. 01–08.
- [7] K. Eykholt, T. Lee, D. Schales, J. Jang, and I. Molloy, "URET: Universal robustness evaluation toolkit (for evasion)," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 3817–3833.
- [8] I. Goodfellow, "NIPS 2016 tutorial: Generative adversarial networks," 2017, *arXiv:1701.00160*.
- [9] I. Goodfellow, P. McDaniel, and N. Papernot, "Making machine learning robust against adversarial inputs," *Commun. ACM*, vol. 61, no. 7, pp. 56–66, 2018.
- [10] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2014, pp. 1–9.
- [11] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [12] D. Han et al., "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2632–2647, Jun. 2021.
- [13] L. Han, Y. Sheng, and X. Zeng, "A packet-length-adjustable attention model based on bytes embedding using flow-WGAN for smart cybersecurity," *IEEE Access*, vol. 7, pp. 82913–82926, 2019.
- [14] K. He, D. D. Kim, and M. R. Asghar, "Adversarial machine learning for network intrusion detection systems: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 538–566, 1st Quart., 2023.
- [15] O. Ibitoye, O. Shafiq, and A. Matrawy, "Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.
- [16] B. Ingre and A. Yadav, "Performance analysis of NSL-KDD dataset using ANN," in *Proc. Int. Conf. Signal Process. Commun. Eng. Syst.*, Jan. 2015, pp. 92–96.
- [17] S. Kulinski and D. Inouye, "Towards explaining distribution shifts," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 17931–17952.
- [18] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, vol. 2, Feb. 2017, pp. 253–262.
- [19] Z. Lin, A. Jain, C. Wang, G. Fanti, and V. Sekar, "Using GANs for sharing networked time series data: Challenges, initial promise, and open questions," in *Proc. ACM IMC*, 2020, pp. 464–483.
- [20] J. X. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," 2020, *arXiv:2005.05909*.
- [21] M. Nasr, A. Bahramali, and A. Houmansadr, "Defeating DNN-based traffic analysis systems in real-time with blind adversarial perturbations," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2705–2722.
- [22] M. Nasr, A. Houmansadr, and A. Mazumdar, "Compressive traffic analysis: A new paradigm for scalable traffic analysis," in *Proc. ACM SIGSAC*, 2017, pp. 2053–2069.
- [23] G. Peyré et al., "Computational optimal transport," Center Res. Econ. Statist. Work. Papers (2017-86), 2017, *arXiv:1803.00567v4*.
- [24] X. Peng, W. Huang, and Z. Shi, "Adversarial attack against DoS intrusion detection: An improved boundary-based method," in *Proc. IEEE 31st Int. Conf. Tools Artif. Intell. (ICTAI)*, Nov. 2019, pp. 1288–1295.
- [25] M. Rigaki and S. Garcia, "Bringing a GAN to a knife-fight: Adapting malware communication to avoid detection," in *Proc. IEEE SPW*, May 2018, pp. 70–75.
- [26] A. Sadeghzadeh, S. Shiravi, and R. Jalili, "Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1962–1976, Jun. 2021.
- [27] I. Sharafaldin, A. Lashkari, and A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 1–9.
- [28] Y. Sharon, D. Berend, Y. Liu, A. Shabtai, and Y. Elovici, "TANTRA: Timing-based adversarial network traffic reshaping attack," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3225–3237, 2022.
- [29] S. Tan, X. Zhong, Z. Tian, and Q. Dong, "Sneaking through security: Mutating live network traffic to evade learning-based nids," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 2295–2308, Sep. 2022.
- [30] M. Usama, M. Asim, S. Latif, J. Qadir, and A. Al-Fuqaha, "Generative adversarial networks for launching and thwarting adversarial attacks on network intrusion detection systems," in *Proc. 15th Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, 2019, pp. 78–83.
- [31] K. Wang et al., "BARS: Local robustness certification for deep learning based traffic analysis systems," in *Proc. NDSS*, 2023, pp. 1–18.
- [32] P. Wang, S. Li, F. Ye, Z. Wang, and M. Zhang, "PacketCGAN: Exploratory study of class imbalance for encrypted traffic classification using CGAN," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–7.
- [33] D. Wu, B. Fang, J. Wang, Q. Liu, and X. Cui, "Evading machine learning botnet detection models via deep reinforcement learning," in *Proc. IEEE ICC*, May 2019, pp. 1–6.
- [34] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional gan," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–11.
- [35] Y. Yin, Z. Lin, M. Jin, G. Fanti, and V. Sekar, "Practical GAN-based synthetic IP header trace generation using netshare," in *Proc. ACM SIGCOMM*, 2022, pp. 458–472.
- [36] N. Moustafa and J. Slay, "The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Secur. J., Global Perspective*, vol. 25, nos. 1–3, pp. 18–31, 2016.
- [37] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 7–12.
- [38] X. Jiang et al., "NetDiffusion: Network data augmentation through protocol-constrained traffic generation," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 1, pp. 1–32, 2024.



Minxiao Wang received the B.S. degree in electronic and communication engineering and the M.S. degree in electronic and information engineering from the Civil Aviation University of China in 2014 and 2018, respectively. He is currently pursuing the Ph.D. degree with the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University, Carbondale, IL, USA. His research interests include developing machine learning and deep learning models for diverse applications related to action recognition, object detection, networks, and intrusion detection.



Ning Yang (Member, IEEE) received the M.S. degree in computer engineering from the University of Massachusetts Amherst, USA, in 2006, and the Ph.D. degree in computer engineering from Southern Illinois University, Carbondale, IL, USA, in 2020. She is currently an Assistant Professor with the School of Computing, Information Technology Program, Southern Illinois University. Her research interests include network security, the Internet of Things, and machine learning.



Nicolas J. Forcade-Perkins received the B.S. degree in computer engineering and mathematics from Southern Illinois University, Carbondale, IL, USA, in 2023. He is currently pursuing the M.S. degree in data science with The University of Texas at Austin, USA. He is an Applications Engineer in power electronics designing custom power modules for data centers with Texas Instruments. His research interests include image recognition systems and predictive analytics.



Ning Weng (Senior Member, IEEE) received the B.S. degree in electrical engineering from Huazhong University of Science and Technology, China, in 1996, the M.S. degree in electrical and computer engineering from the University of Central Florida, Orlando, FL, USA, in 2000, and the Ph.D. degree in electrical and computer engineering from the University of Massachusetts, Amherst, MA, USA, in 2005. He is currently a Full Professor with the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University, Carbondale, IL, USA. He is engaged in research and teaching in the areas of computer networks and security. His research interests include scalable system design for deep packet inspection and many-field packet classification, quality of information modeling, and deep learning models for network intrusion detection. He has been active as a Program Committee Member of several professional conferences, including IEEE INFOCOM and ACM SAC. At Southern Illinois University, he has received three times for Outstanding Teacher Awards from the Department of Electrical and Computer Engineering.